



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

Lập trình Hướng Đối tượng Object Oriented Programming

\$6 Kế thừa





Nội dung

- Khái niệm
- Biểu diễn kế thừa
- Nguyên lý kế thừa
- Khởi tạo và hủy bỏ đối tượng lớp con kế thừa





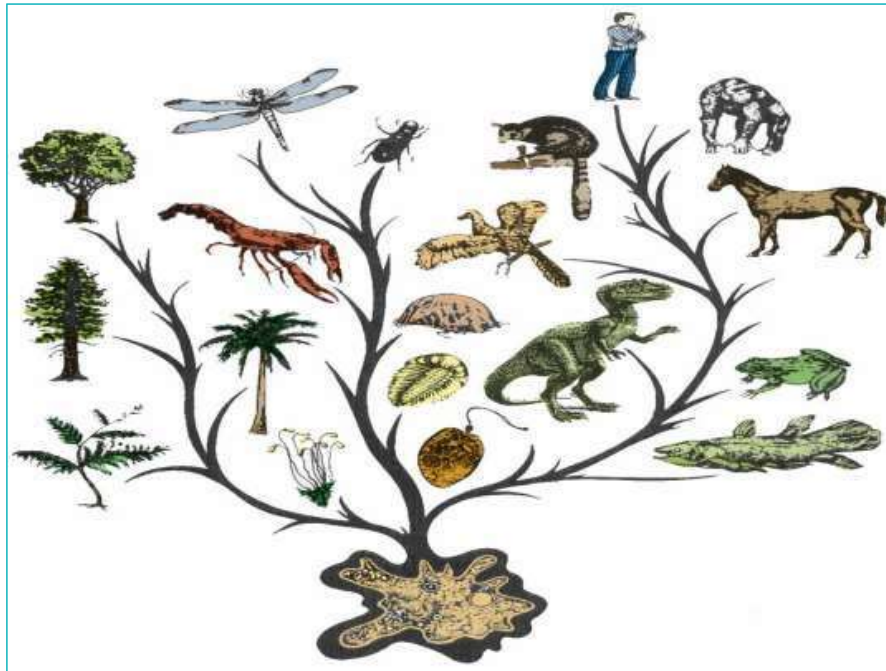
ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

1. Kế thừa - Inheritance



Kế thừa



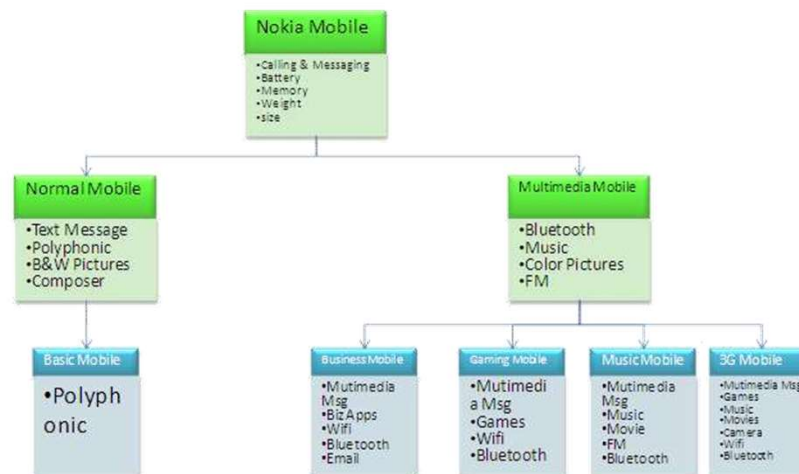
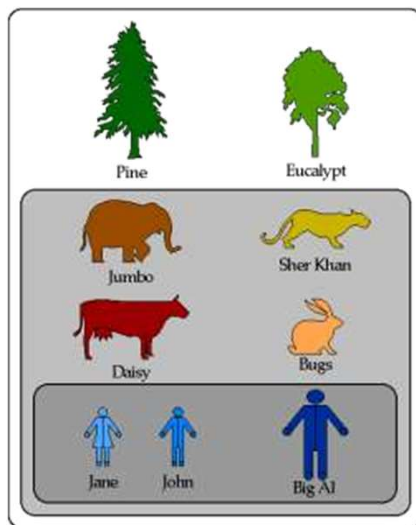
“Xây dựng các lớp mới có sẵn các đặc tính của lớp cũ, đồng thời chia sẻ hay mở rộng các đặc tính sẵn có”

Bản chất kế thừa

- Phát triển lớp đối tượng mới dựa trên lớp đã có
- VD:
 - Lớp Person
 - properties: họ tên, NS
 - methods: ăn, chơi
 - Lớp Student:
 - kế thừa attr ? method ?
 - bổ sung attr ? method ?

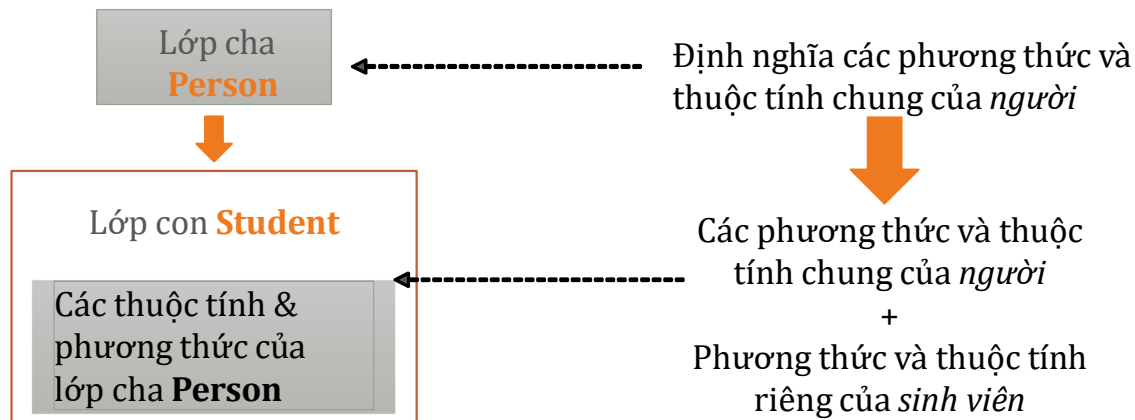
Bản chất kế thừa

- Chính là nguyên lý phân cấp trong trừu tượng hóa



Ví dụ

- Lớp cũ
 - lớp cha (parent, superclass); lớp cơ sở (base class)
- Lớp mới
 - lớp con (child, subclass); lớp dẫn xuất (derived class)



Mối quan hệ kế thừa

- Lớp con và lớp cha có tính tương đồng
 - Lớp Student kế thừa từ lớp Person
 - Một student là một person

Lớp cha
Person

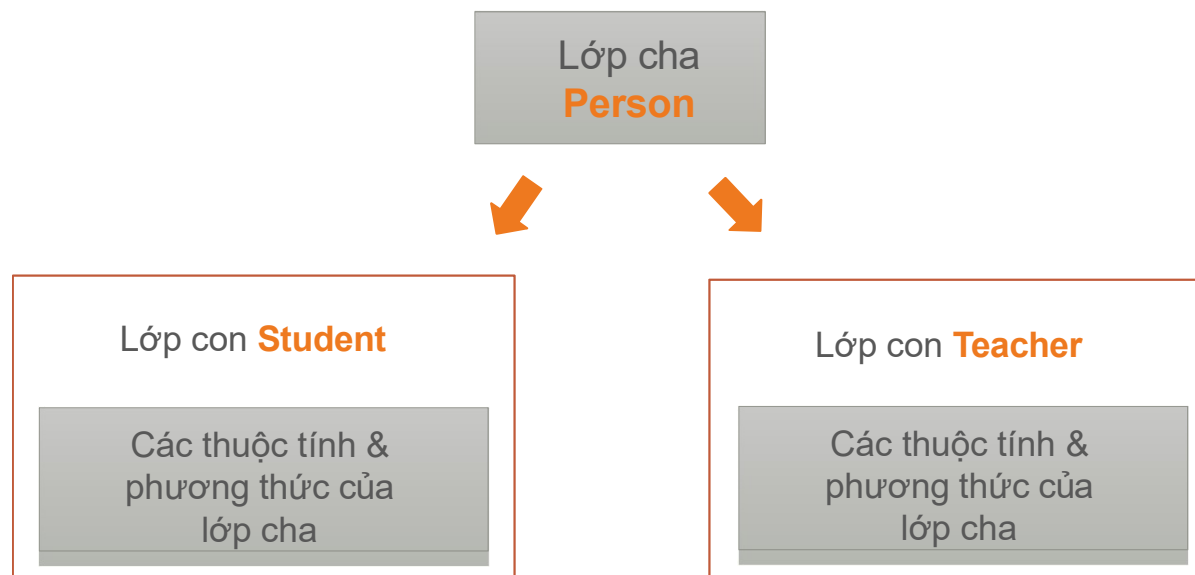


Lớp con **Student**

Các thuộc tính &
phương thức của
lớp cha Người

Mối quan hệ kế thừa

- Các lớp Student và Teacher có quan hệ **là (is-a)** với lớp person
=> Student và Teacher có đặc tính, hành vi thông thường của person



Mối quan hệ kế thừa

- Lớp con
 - Là một **loại**/kiểu (**is-a-kind-of**) của lớp cha
 - Kế thừa các thành phần dữ liệu và các hành vi của lớp cha
 - Chi tiết hóa cho phù hợp với mục đích sử dụng mới
 - Extension: Thêm các thuộc tính/hành vi mới
 - Redefinition (Method Overriding): Chỉnh sửa lại các hành vi kế thừa từ lớp cha

Cú pháp (Java)



- Cú pháp (Java):
`<Lớp con> extends <Lớp cha>`

- Ví dụ

```
class Nguoi {  
    String name; int age;  
}  
class SinhVien extends Nguoi  
{  
    int studentId;  
}
```

Lớp con *mở rộng* các đặc tính của lớp cha

Lớp cha **Nguoi**
name, age



Lớp con **SinhVien**
studentId

name, age

Cú pháp (C++/C#)



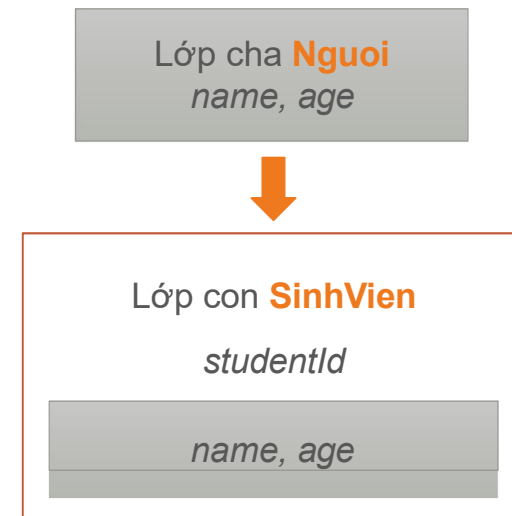
- Cú pháp (Java):

<Lớp con> : <Lớp cha>

- Ví dụ

```
class Nguoi {  
    String name; int age;  
};  
class SinhVien : Nguoi {  
    int studentId;  
};
```

– Lớp con là một đối tượng lớp cha



Bản chất kế thừa

- Là một kỹ thuật tái sử dụng mã nguồn
 - Tái sử dụng mã nguồn thông qua lớp cha
- Ví dụ
 - Lớp Student tái sử dụng được các thuộc tính như tên, tuổi... và các phương thức của lớp Person.

C# không hỗ trợ đa kế thừa (mỗi lớp kế thừa chỉ có một lớp cơ sở)

Kế thừa và kết tập

- Kế thừa
 - Tái sử dụng thông qua lớp
 - Tạo lớp mới bằng cách phát triển lớp đã có sẵn
 - Quan hệ “là một loại” (is a kind of)
- Kết tập
 - Tái sử dụng thông qua đối tượng
 - Tạo ra tham chiếu đến các đối tượng của các lớp có sẵn trong lớp mới
 - Quan hệ “là một phần” (is a part of)



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

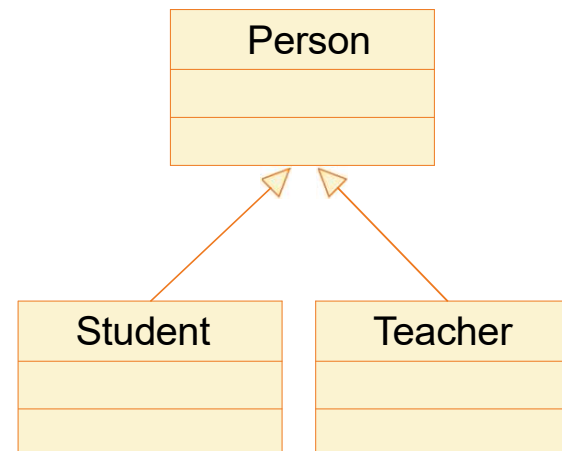
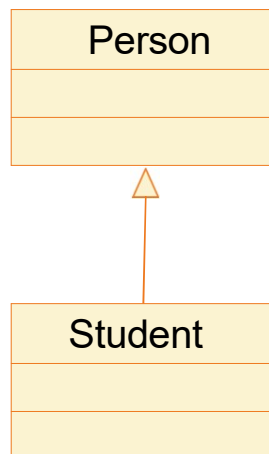
KHOA TOÁN – TIN

2. Biểu diễn quan hệ kế thừa trong biểu đồ lớp



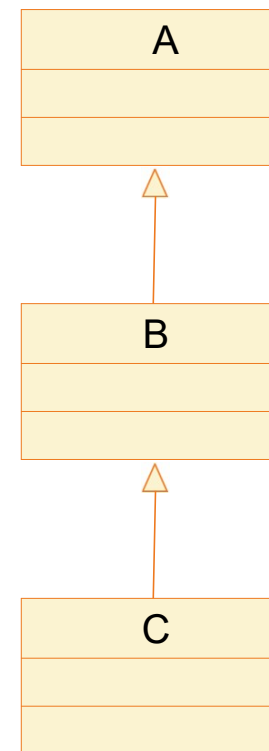
Cây phân cấp kế thừa

- Mũi tên với tam giác rỗng ở đầu



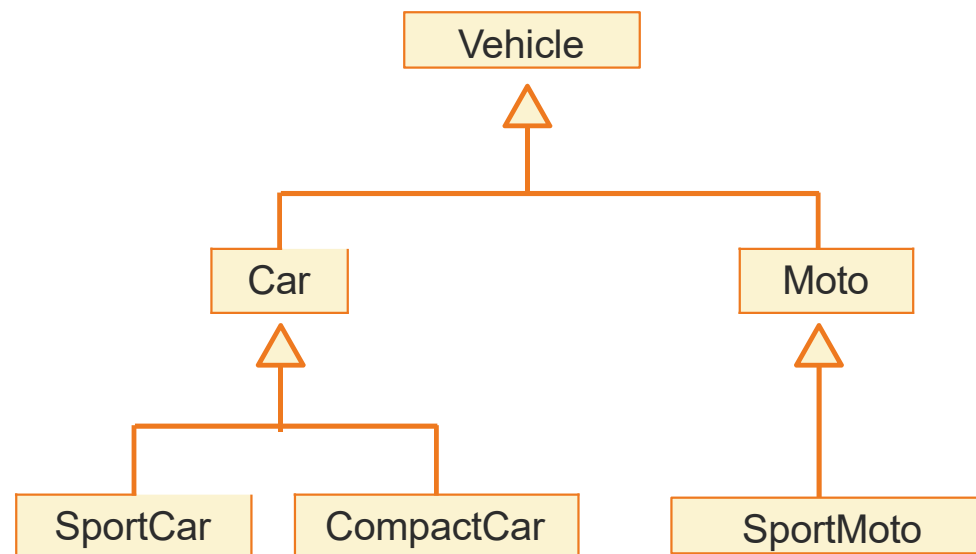
Cây phân cấp kế thừa

- Cấu trúc phân cấp hình cây, biểu diễn mối quan hệ kế thừa giữa các lớp.
 - Dẫn xuất trực tiếp
 - B dẫn xuất trực tiếp từ A
 - Dẫn xuất gián tiếp
 - C dẫn xuất gián tiếp từ A



Cây phân cấp kế thừa

- Các lớp con có cùng lớp cha gọi là các lớp anh chị em (siblings)



Lớp Object



Java

- Lớp **Object** là lớp gốc trên cùng của tất cả các cây phân cấp kế thừa
 - Nếu một lớp không được định nghĩa là lớp con của một lớp khác thì mặc định nó là lớp con trực tiếp của lớp Object.
 - Được định nghĩa trong package chuẩn java.lang

C#

- Tương tự

Lớp Object

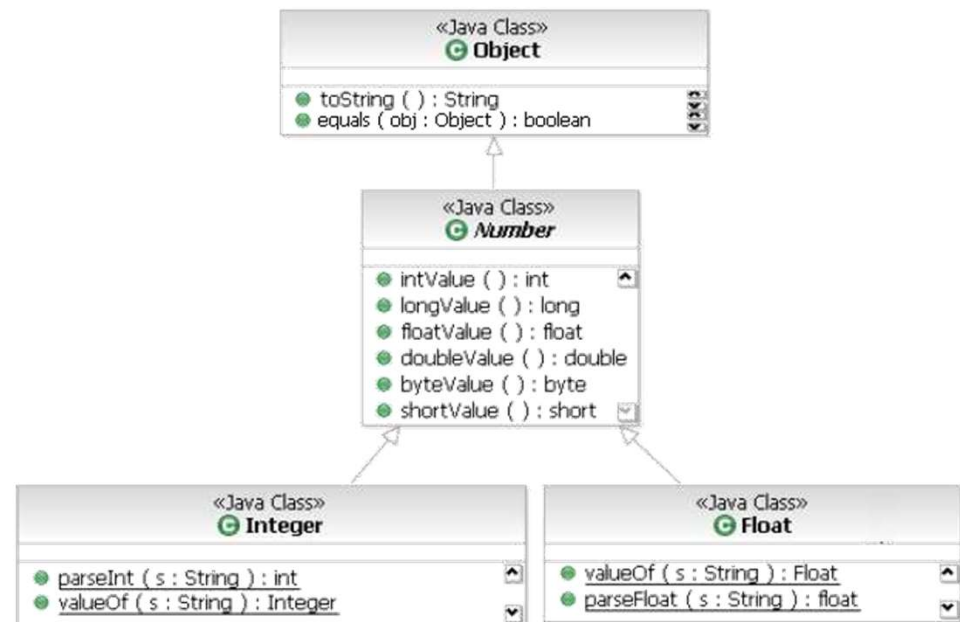


Java

- Chứa một số phương thức hữu ích kế thừa lại cho tất cả các lớp
- VD: toString(), equals()...

C#

- Tương tự





ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

3. Nguyên lý kế thừa



Nguyên lý kế thừa

- Lớp con có thể thừa kế được gì từ lớp cha? (có Phụ thuộc cài đặt của ngôn ngữ)
- Nói chung:
 - Class con kế thừa tất cả thành viên của lớp cha (và lớp cao hơn)
 - **public** thành viên có thể truy cập được bởi code bất kỳ đâu => Kế thừa
 - **protected** phương thức, thuộc tính chỉ có thể truy cập, gọi bởi các dòng code cùng lớp hoặc các lớp kế thừa nó => Kế thừa
 - **private** phương thức, thuộc tính, trường khai báo với private chỉ có thể truy cập, gọi bởi các dòng code cùng lớp => Không kế thừa

Nguyên lý kế thừa

■ Java

- Lớp con có thể thừa kế được gì từ lớp cha?
 - Kế thừa được các thành viên được khai báo là public và **protected** của lớp cha.
 - Không kế thừa được các thành viên private.

*Thành viên **protected** trong lớp cha được truy cập trong:*

- *Các thành viên lớp cha*
- *Các thành viên lớp con*
- *Các thành viên các lớp cùng thuộc 1 package với lớp cha*

Nguyên lý kế thừa

- Java: phạm vi kế thừa

	public	protected	mặc định	private
Cùng lớp	✓	✓	✓	✓
Lớp bất kỳ cùng gói	✓	✓	✓	✗
Lớp con khác gói	✓	✓	✗	✗
Lớp bất kỳ khác gói	✓	✗	✗	✗

Nguyên lý kế thừa

■ Java

- Các phương thức không được kế thừa:
 - Các phương thức khởi tạo và hủy
 - Làm nhiệm vụ khởi đầu và gỡ bỏ các đối tượng
 - Chúng chỉ biết cách làm việc với từng lớp cụ thể

■ C#: tương tự

- Khi khai báo constructor lớp cha => lớp con cũng phải khai báo constructor
- Constructor lớp con: chỉ rõ gọi constructor nào của lớp cha

Ví dụ (C#)

```
// Base class
public class Person
{
    public string Name { get; set; } = "";
    public int Age { get; set; }

    public void DisplayPersonInfo()
    {
        Console.WriteLine($"Name: {Name}, Age: {Age}");
    }
}
```

Ví dụ (C#)

```
// Derived class
public class Teacher : Person
{
    public int EmployeeID { get; set; }
    public Teacher(...){
        base.Name = "Thang"; base.Age = 50;
        EmployeeID = 0;
    }
    public void DisplayEmployeeInfo(){
        base.DisplayPersonInfo(); //Accessing members of base class
        Console.WriteLine($"Employee ID: {EmployeeID}");
    }
}
```

```
Teacher t1 = new Teacher();
t1.EmployeeID = 12121;
t1.Name = "Thang";
t1.DisplayEmployeeInfo();
```

Ví dụ 2 (C#)

```
// Base class
public class Person
{
    protected string Name { get; set; }
    protected int Age { get; set; }
    public Person(string name, int age)
    {
        Name = name;
        Age = age;
    }
    public void DisplayPersonInfo()
    {
        Console.WriteLine($"Name: {Name}, Age: {Age}");
    }
}
```

Ví dụ 2 (C#)

// Derived class

```
public class Teacher : Person
{
    public int EmployeeID { get; set; }
    public Teacher(string name, int age, int employeeID) : base(name, age)
    {
        EmployeeID = employeeID;
    }
    public void DisplayEmployeeInfo() {
        DisplayPersonInfo(); //Accessing protected members of base class
        Console.WriteLine($"Employee ID: {EmployeeID}");
    }
}
```

```
public class Person {
    protected string Name { get; set; }
    protected int Age { get; set; }
    ...
    public void DisplayPersonInfo() {
        Console.WriteLine($"Name: {Name}, Age: {Age}");
    }
}
```

```
Teacher t1 = new Teacher("Thang", 36, 12345);
int curId = t1.EmployeeID; // Accessing members of derived class
t1.DisplayPersonInfo(); // Accessing protected members of base class
string curName = t1.Name; //Error: 'Person.Name' is inaccessible due to its protection level
```

Ví dụ 1 (java)

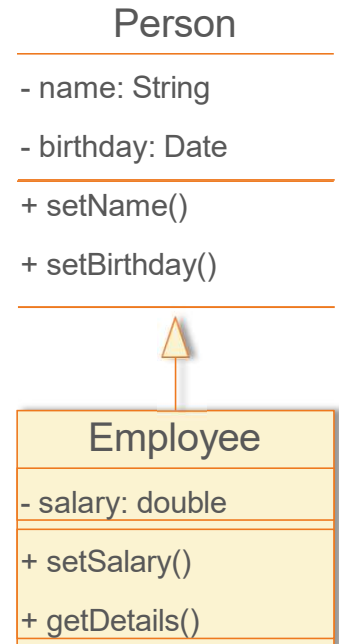
```
public class TuGiac {  
    protected Diem d1, d2, d3, d4;  
    public void setD1(Diem _d1) {d1=_d1;}  
    public Diem getD1(){return d1;}  
    public void printTuGiac(){...}  
}  
  
public class HìnhVuong extends TuGiac {  
    public HìnhVuong(){  
        d1 = new Diem(0,0); d2 = new Diem(0,1);  
        d3 = new Diem(1,0); d4 = new Diem(1,1);  
    }  
}  
  
public class Test{  
    public static void main(String  
        args[]){ HìnhVuong hv = new  
        HìnhVuong(); hv.printTuGiac();  
    }  
}
```

Sử dụng các thành phần
protected của lớp cha
trong lớp con

Gọi phương thức *public* của lớp
cha trong đối tượng lớp con

Ví dụ 2

```
class Person {  
    private String name;  
    private Date birthday;  
    public String getName() {return name;}  
}  
class Employee extends Person {  
    private double salary;  
    public boolean setSalary(double sal) {  
        salary = sal;  
        return true;  
    }  
    public String getDetail() {  
        String s = name + ", " + birthday + ", " + salary; //ERROR  
    }  
}
```

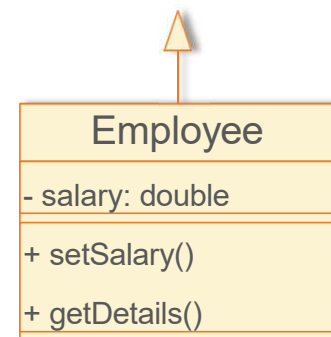


Ví dụ 2 (tiếp)

```
public class Test{  
    public static void main(String  
        args[]){ Employee e = new  
        Employee(); e.setName("John");  
        e.setSalary(3.0);  
    }  
}
```

Person

- name: String
- birthday: Date
- + setName()
- + setBirthday()



Ví dụ 3

- Cùng gói
- Khác gói

```
package abc;  
public class Person {  
    Date birthday;  
    String name;  
}
```

```
package abc.Person;  
public class Employee extends Person {  
    double salary;  
    public String getDetail() {  
        String s;  
        s = name + ", " + birthday + ", " + salary;  
        return s;  
    }  
}
```

Ví dụ 3

```
package abc;
public class Person {
    protected Date birthday;
    protected String name;
}

package abc.Person;
public class Employee extends Person {
    double salary;
    public String getDetail()
    { String s;
      s = name + ", " + birthday + ", " + salary;
      return s;
    }
}
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

4. Khởi tạo và hủy bỏ đối tượng trong kế thừa



Khởi tạo và hủy bỏ đối tượng

■ Khởi tạo và hủy bỏ đối tượng trong kế thừa

• Khởi tạo - constructor

- Lớp cha được khởi tạo trước lớp con
- Các phương thức khởi tạo của lớp con luôn gọi Phương thức khởi tạo của lớp cha ở câu lệnh đầu tiên
 - Tự động gọi (implicit): Khi lớp cha có constructor ngầm định
 - Gọi tường minh: Khi lớp cha có constructor

```
public HinhVuong(Diem d1, ...){  
    super(d1, d2, d3, d4);  
}
```

*Java: Sử dụng từ khóa **super***

```
public Teacher(string name, int age, int id):base(name, age){  
    EmployeeID = id;  
}
```

*C#: Sử dụng từ khóa **base***

• Hủy bỏ đối tượng

- Ngược lại so với khởi tạo đối tượng

Gọi phương thức của lớp cha

- Tái sử dụng các đoạn mã của lớp cha trong lớp con
 - Gọi phương thức khởi tạo
 - Ngầm định
 - Tường minh:
 - Bắt buộc nếu lớp cha không có phương thức khởi tạo mặc định
 - Gọi phương thức constructor mong muốn từ lớp cha (Gọi đầu tiên)
 - Gọi các phương thức của lớp cha
 - Lớp con có thể gọi các Phương thức của lớp cha (tham chiếu với từ khóa base - C# hoặc super - Java)

`super.DisplayPersonInfo();`

`base.DisplayPersonInfo();`

Ví dụ (C#)

- Hàm khởi tạo của lớp cơ sở chạy trước, xong đến hàm khởi tạo của lớp kế thừa.

```
class A {  
    public A() {  
        Console.WriteLine("A Init");  
    }  
}
```

```
class B : A {  
    public B()  
    {  
        Console.WriteLine("B Init");  
    }  
}
```

```
B obj1 = new B();  
// output:  
// A Init  
// B Ini
```

Ví dụ (C#)

- Nếu constructor lớp cơ sở có tham số, hoặc overloading => Constructor lớp kế thừa phải chỉ định sẽ khởi chạy constructor (và truyền tham số) nào của lớp cơ sở

```
class A {  
    public A(string mgs) {  
        Console.WriteLine("A Init" + mgs);  
    }  
}
```

```
class B : A {  
    public B(string abc) : base(abc) {  
        Console.WriteLine("B Init");  
    }  
}
```

```
B obj1 = new B("!ABC");  
// Output:  
// A Init!ABC  
// B Init
```

Chuyển kiểu - tương thích về kiểu khi sử dụng lớp

- Trong chuỗi kế thừa ví dụ lớp A là lớp cơ sở, kế thừa bởi lớp B, lớp B lại là lớp cơ sở kế thừa bởi C
 - Có thể chuyển kiểu từ C -> B -> A. Bạn có thể chuyển kiểu một cách tường minh (viết tên kiểu muốn chuyển trong () trước đối tượng), hay ngầm định
 - Chiều ngược lại thì không thể

```
C c = new C();  
a = (A)c;           // chuyển kiểu tường minh  
a = c;              // ngầm định  
a = new C();        // ngầm định  
B b = c;             // ngầm định  
c = new A();         // lỗi - không thể chuyển kiểu thuận cây kế thừa
```

Chuyển kiểu - tương thích về kiểu khi sử dụng lớp

- Toán tử 'is'

```
Teacher t1 = new Teacher();  
t1.EmployeeID = 12121;  
t1.Name = "Thang";  
t1.DisplayEmployeeInfo();
```

```
bool isPerson = t1 is Person;  
bool isTeacher = t1 is Teacher;
```

Lớp sealed

- Không cho phép kế thừa
- Từ khóa `sealed`

```
sealed class A
{
    . . .
}

class B : A // cannot derive from sealed class
{
}
```

